# 기타 예제들 (Blake, Fibonacci, State Machine) 분석

## 개요

Stwo 프로젝트는 Poseidon 해시 함수 외에도 다양한 계산 패턴을 Circle STARK로 증명하는 예제들을 제공합니다. 이 문서는 Blake 해시 함수, Wide Fibonacci 수열, 그리고 State Machine 예제를 분석하여 각각의 특성과 Circle STARK 활용 방식, 제약 조건 설계, 성능 특성을 상세히 설명합니다.

## Blake 해시 함수 분석

### Blake2s/Blake3 구조적 특성

Blake 해시 함수는 Poseidon과는 다른 접근 방식을 보여주는 중요한 예제입니다:

```
// Blake 해시 함수 기본 구조

const STATE_SIZE: usize = 16;

const MESSAGE_SIZE: usize = 16;

const N_FELTS_IN_U32: usize = 2;

const N_ROUNDS: usize = 10;

const ROUND_LOG_SPLIT: [u32; 2] = [3, 1]; // 8 + 2 라운드로 분할
```

### Circle STARK 적용 방식

#### 1. 모듈화된 컴포넌트 구조

Blake 구현은 여러 컴포넌트로 분할되어 있습니다:

```
pub struct BlakeComponents {
scheduler_component: BlakeSchedulerComponent, // 스케줄링 로직
round_components: Vec<BlakeRoundComponent>, // 라운드 연산
xor12: xor12::XorTableComponent<12, 4>, // XOR 룩업 테이블들
xor9: xor9::XorTableComponent<9, 2>,
xor8: xor8::XorTableComponent<8, 2>,
xor7: xor7::XorTableComponent<7, 2>,
xor4: xor4::XorTableComponent<4, 0>,
}
```

**모듈화의 이점**:

- **복잡도 관리**: 각 컴포넌트별 독립적 최적화
- **재사용성**: XOR 테이블의 다중 활용
- **병렬 처리**: 컴포넌트 간 독립적 증명 생성

### 2. XOR 연산 룩업 테이블

Blake의 핵심 특징은 비트 연산을 필드 연산으로 변환하는 것입니다:

```
#[derive(Default)]
struct XorAccums {
xor12: xor12::XorAccumulator<12, 4>, // 12비트 XOR, 4비트 분할
xor9: xor9::XorAccumulator<9, 2>, // 9비트 XOR, 2비트 분할
xor8: xor8::XorAccumulator<8, 2>, // 8비트 XOR, 2비트 분할
```

```
xor7: xor7::XorAccumulator<7, 2>, // 7비트 XOR, 2비트 분할
xor4: xor4::XorAccumulator<4, 0>, // 4비트 XOR, 분할 없음
}
```

**XOR 테이블 활용 방식**:

- **전처리된 테이블**: 모든 가능한 XOR 연산 결과 미리 계산
- **룩업 최적화**: 빈번한 XOR 연산을 테이블 조회로 대체
- **메모리 트레이드오프**: 계산 복잡도를 메모리 사용량으로 교환

### 3. u32 필드 원소 분할

Blake는 32비트 정수를 두 개의 필드 원소로 분할합니다:

```
/// u32를 두 개의 필드 원소로 표현하는 유틸리티
#[derive(Clone, Debug)]
struct Fu32<F> {
l: F, // 하위 16비트
h: F, // 상위 16비트
}

/// u32를 2개의 필드 원소로 분할
fn  to_felts(x: &u32x16) -> [PackedBaseField; 2] {
[
unsafe { PackedBaseField::from_simd_unchecked(x & u32x16::splat(0xffff)) },
unsafe { PackedBaseField::from_simd_unchecked(x >> 16) },
```

```
]

}
```

## 제약 조건 설계 분석

### 1. 라운드 함수 제약 조건

Blake의 각 라운드는 복잡한 비트 연산을 포함합니다:

```
impl BlakeXorElements {

fn  use_relation<E: EvalAtRow>(&self, eval: &mut E, w: u32, values: [&[E::F]; 2]

match  w {

12 => {

eval.add_to_relation(RelationEntry::new(&self.xor12, E::EF::one(), values[0]));

eval.add_to_relation(RelationEntry::new(&self.xor12, E::EF::one(), values[1]));

}

// 다른 비트 폭에 대한 처리...

}

}

}
```

**제약 조건 특성**:

- **비트 연산 추상화**: XOR, 회전, 시프트 연산을 필드 연산으로 변환
- **룩업 기반 검증**: 복잡한 비트 연산을 테이블 조회로 단순화
- **모듈러 설계**: 각 연산 타입별 독립적 제약 조건

### 2. 스케줄러 제약 조건

메시지 스케줄링 로직도 별도 컴포넌트로 구현됩니다:

```
pub struct BlakeSchedulerEval {
pub log_size: u32,
pub blake_lookup_elements: BlakeElements,
pub round_lookup_elements: RoundElements,
pub claimed_sum: SecureField,
}
```

## 성능 특성 및 최적화

### 1. 전처리된 테이블 활용

```
fn preprocessed_xor_columns() -> [PreProcessedColumnId; 15] {
[
XorTable::new(12, 4, 0).id(),
XorTable::new(12, 4, 1).id(),
XorTable::new(12, 4, 2).id(),
// ... 다른 XOR 테이블들
]
}
```

**최적화 효과**:

- **증명 시간 단축**: 런타임 계산을 전처리로 대체

- **메모리 효율성**: 공유 테이블을 통한 중복 제거
- **검증 속도**: 빠른 테이블 조회 기반 검증

**2. 라운드 분할 최적화**

```
const ROUND_LOG_SPLIT: [u32; 2] = [3, 1]; // 8 + 2 라운드
```

**분할 전략의 이점**:

- **병렬 처리**: 라운드 그룹별 독립적 처리
- **메모리 관리**: 작은 단위로 분할하여 메모리 사용량 최적화
- **확장성**: 다양한 라운드 수에 대한 유연한 대응

# Wide Fibonacci 수열 분석

## 수학적 구조

Wide Fibonacci는 전통적인 피보나치 수열을 확장한 형태입니다:

```
/// 각 행이 길이 N인 별도의 피보나치 수열을 포함
#[derive(Clone)]
pub struct WideFibonacciEval<const N: usize> {
pub log_n_rows: u32,
}
```

## Circle STARK 적용 방식

**1. 제곱 기반 점화식**

일반적인 피보나치와 달리 제곱 연산을 사용합니다:

```
impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
for _ in 2..N {
let c = eval.next_trace_mask();
eval.add_constraint(c.clone() - (a.square() + b.square())); // c = a² + b²
a = b;
b = c;
}
eval
}
}
```

**수학적 특성**:

- **비선형 점화식**: $c = a^2 + b^2$ 관계
- **필드 친화적**: 제곱 연산은 필드에서 효율적
- **병렬 처리**: 각 행이 독립적인 수열

**2. 트레이스 생성 최적화**

```
pub fn generate_trace<const N: usize>(
log_size: u32,
```

```
inputs: &[FibInput],
) -> ColumnVec<CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>> {
let mut trace = (0..N)
.map(|_| Col::<SimdBackend, BaseField>::zeros(1 << log_size))
.collect_vec();
for (vec_index, input) in inputs.iter().enumerate() {
let mut a = input.a;
let mut b = input.b;
trace[0].data[vec_index] = a;
trace[1].data[vec_index] = b;
trace.iter_mut().skip(2).for_each(|col| {
(a, b) = (b, a.square() + b.square());
col.data[vec_index] = b;
});
}
// Circle 도메인으로 변환
let domain = CanonicCoset::new(log_size).circle_domain();
trace.into_iter()
.map(|eval| CircleEvaluation::new(domain, eval))
.collect_vec()
}
```

## 성능 특성

**1. SIMD 최적화**

```
pub struct FibInput {
a: PackedBaseField, // 16개 값을 동시 처리
b: PackedBaseField,
}
```

**벡터화 이점**:

- **16배 병렬성**: PackedBaseField를 통한 SIMD 활용
- **메모리 효율성**: 연속된 메모리 접근
- **캐시 친화적**: 지역성을 고려한 데이터 배치

**2. 제약 조건 복잡도**

**복잡도 분석**:

- **제약 조건 수**: O(N) - 수열 길이에 비례
- **차수**: 2차 (제곱 연산으로 인해)
- **메모리 사용량**: O(N × 2^log_size)

# State Machine 분석

## 상태 전이 모델

State Machine 예제는 2차원 격자에서의 상태 전이를 모델링합니다:

```
pub type State = [M31; STATE_SIZE];
const STATE_SIZE: usize = 2;
```

```
// 상태 전이: (x, y) → (x+1, y) 또는 (x, y) → (x, y+1)
```

## Circle STARK 적용 방식

### 1. 이중 컴포넌트 구조

```
pub struct StateMachineComponents {
component0: StateMachineOp0Component, // X축 이동 (op=0)
component1: StateMachineOp1Component, // Y축 이동 (op=1)
}
```

**컴포넌트별 역할**:

- **Op0 컴포넌트**: X축 방향 상태 전이 (x → x+1)
- **Op1 컴포넌트**: Y축 방향 상태 전이 (y → y+1)
- **독립적 증명**: 각 연산 타입별 별도 증명 생성

### 2. 룩업 기반 상태 검증

```
relation!(StateMachineElements, STATE_SIZE);

pub struct StateTransitionEval {
pub log_n_rows: u32,
pub lookup_elements: StateMachineElements,
pub claimed_sum: SecureField,
```

```
}
```

**상태 전이 제약 조건**:

- **초기 상태 룩업**: 전이 시작점 검증
- **최종 상태 룩업**: 전이 종료점 검증
- **전이 규칙**: 허용된 상태 변화만 인정

### 3. 상호작용 트레이스

```
pub fn gen_interaction_trace(
trace: &[CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>],
op: usize,
lookup_elements: &StateMachineElements,
) -> (ColumnVec<CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>>, Sec
// 상태 전이에 대한 룩업 검증 생성
}
```

## 제약 조건 설계

### 1. 상태 일관성 검증

```
// 상태 전이 제약 조건 예시
let initial_state_comb: QM31 = lookup_elements.combine(&initial_state);
let final_state_comb: QM31 = lookup_elements.combine(&final_state);
```

```
assert_eq!(
(x_axis_claimed_sum + y_axis_claimed_sum) * initial_state_comb * final_state_com
final_state_comb - initial_state_comb
);
```

**검증 메커니즘**:

- **상태 조합**: 다중 필드 원소를 단일 값으로 결합
- **전이 합계**: 모든 상태 전이의 누적 검증
- **일관성 체크**: 초기 상태와 최종 상태의 관계 검증

### 2. 연산별 분리 검증

```
impl FrameworkEval for StateTransitionEval {
fn  evaluate<E: EvalAtRow>(&self, mut  eval: E) -> E {
// 각 연산 타입별 독립적 제약 조건
eval_state_transition_constraints(&mut  eval, &self.lookup_elements);
eval
}
}
```

## 성능 특성 및 확장성

### 1. 로그 복잡도 검증

**복잡도 분석**:

- **증명 크기**: O(log n) - 상태 수에 대해 로그 복잡도

- **검증 시간**: O(log n) - 빠른 검증
- **메모리 사용량**: 상태 크기에 선형 비례

**2. 병렬 처리 가능성**

```
// 병렬 상태 전이 처리
for vec_index in 0..(1 << (log_size - LOG_N_LANES)) {
// 16개 상태를 동시에 처리
let packed_state = PackedBaseField::from_array(/* ... */);
}
```

## 예제별 성능 비교 분석

### 1. 계산 복잡도 비교

| 예제 | 제약 조건 수 | 차수 | 메모리 사용량 | 특징 |
|---|---|---|---|---|
| Poseidon | O(라운드 수) | 5차 | 중간 | 해시 함수, S-box 연산 |
| Blake | O(라운드 수 × XOR 수) | 1차 | 높음 | 룩업 테이블 집약적 |
| Fibonacci | O(수열 길이) | 2차 | 낮음 | 단순한 점화식 |
| State Machine | O(전이 수) | 1차 | 낮음 | 상태 전이 검증 |

### 2. 최적화 기법 비교

**Poseidon vs Blake**

- **Poseidon**: 필드 연산 최적화, S-box 효율성

- **Blake**: 룩업 테이블 활용, 비트 연산 추상화

**Fibonacci vs State Machine**

- **Fibonacci**: 단순한 산술 연산, 높은 병렬성
- **State Machine**: 복잡한 상태 관리, 룩업 기반 검증

### 3. 실제 사용 사례별 적합성

**암호학적 프리미티브**

- **Poseidon**: 영지식 증명 친화적 해시
- **Blake**: 기존 시스템과의 호환성

**계산 증명**

- **Fibonacci**: 반복 계산 패턴 증명
- **State Machine**: 복잡한 상태 전이 시스템

## Circle STARK 활용 패턴 분석

### 1. 공통 설계 원칙

**모듈화**

```
// 모든 예제에서 공통적으로 사용되는 패턴
pub type ExampleComponent = FrameworkComponent<ExampleEval>;


impl FrameworkEval for ExampleEval {
fn evaluate<E: EvalAtRow>(&self, eval: E) -> E {
```

```
// 제약 조건 평가 로직
}
}
```

### 룩업 테이블 활용

```
// 공통 룩업 패턴
relation!(ExampleElements, ELEMENT_COUNT);

eval.add_to_relation(RelationEntry::new(
&lookup_elements,
coefficient,
&values,
));
```

## 2. 최적화 전략

### SIMD 벡터화

- **공통점**: 모든 예제에서 PackedBaseField 활용
- **차이점**: 벡터화 수준과 메모리 접근 패턴

### 병렬 처리

- **Rayon 활용**: 작업 분할을 통한 멀티코어 활용
- **독립성**: 컴포넌트 간 독립적 처리 가능

### 메모리 최적화

- **Circle 도메인**: 효율적인 FFT 연산
- **캐시 친화적**: 지역성을 고려한 데이터 배치

## 결론

각 예제는 Circle STARK 기술의 다양한 측면을 보여줍니다:

### 1. 다양성과 유연성

- **Poseidon**: 고차 다항식과 복잡한 필드 연산
- **Blake**: 룩업 테이블과 비트 연산 추상화
- **Fibonacci**: 단순한 점화식과 높은 병렬성
- **State Machine**: 상태 전이와 복잡한 검증 로직

### 2. 공통 최적화 기법

- **SIMD 벡터화**: 모든 예제에서 16배 병렬 처리
- **Circle 도메인**: 효율적인 FFT와 메모리 접근
- **모듈화 설계**: 재사용 가능한 컴포넌트 구조

### 3. 실용적 적용 가능성

- **암호학적 프리미티브**: Poseidon, Blake
- **계산 증명**: Fibonacci, State Machine
- **확장성**: 다양한 계산 패턴으로의 확장 가능

이러한 분석을 통해 Circle STARK 기술이 다양한 계산 패턴에 어떻게 적용될 수 있는지, 그리고 각 패턴별로 어떤 최적화 전략이 효과적인지 명확히 이해할 수 있습니다.